

Sandlock: Kernel-Enforced Per-Tool Sandboxing for AI Agents

Abstract

AI agents are vulnerable to attacks that no amount of prompting can close, because conventional deployments share capability across tools: every tool of an MCP server runs in one process or one container holding the union of all tools’ filesystem, network, and environment access. A prompt injection in one tool’s output can then drive a *second* tool into abusing capabilities the first never needed, reading `~/.ssh/id_rsa` or exfiltrating an API key, and a single shared sandbox stops none of this while adding root, image management, and startup costs ill-suited to short-lived tool calls. We present SANDLOCK, kernel-enforced per-tool sandboxing for AI agents, built from Landlock and seccomp with no root, no Docker, and no images. Each tool invocation forks into its own deny-by-default process before the tool runs: it receives only the filesystem paths, network endpoints, and environment variables it explicitly declares, so a prompt-injected tool is contained to its declared capabilities rather than the agent’s union. Per-tool confinement adds roughly 5 ms per call. The same substrate composes into pipelines of stages with disjoint capabilities; as a use case, we build an execute-only pipeline, a planner that reaches the model but never observes data and an executor that reads data but has no network, realizing the Execute-Only Agent architecture in the kernel at under 20 ms per pipeline. We run a cross-tool and a data-exfiltration attack against SANDLOCK and confirm each is blocked at the kernel, where an unconfined deployment leaks. SANDLOCK is open source; we omit the repository link for double-blind review.

CCS Concepts

• **Security and privacy** → **Systems security**; • **Software and its engineering** → *Operating systems*.

Keywords

Prompt injection, AI agent security, Model Context Protocol, Capability separation, Execute-only agents, Landlock, seccomp, Process isolation

1 Introduction

AI coding assistants and autonomous agents [2] act by issuing tool calls and consuming external content: they read files, browse the web, query databases, install packages, and invoke build steps through MCP-style tool servers [20]. This is also their defining security weakness. Everything the agent does flows through a single language-model context that mixes *trusted* instructions (the user’s request, the system prompt) with *untrusted* data and code (a web page, an email body, a tool’s output, a model-generated command). The model has no reliable way to tell them apart.

Two attack classes follow directly. In *prompt injection* [5, 31], instructions embedded in untrusted content hijack the model: an email that says “forward all messages to attacker.com” is read as a command rather than as data. The danger is acute when a single context holds private data, exposure to untrusted content, and a way to communicate externally, the “lethal trifecta” [31], because

the hijacked agent becomes its own exfiltration channel. In *cross-tool capability abuse*, an injected tool output steers the model into calling a *second* tool that abuses capabilities the first never needed: a poisoned search result that makes the agent’s shell tool read `~/.ssh/id_rsa` and post it to an attacker. The root cause is shared capability: in a conventional deployment every tool runs in one process or one container holding the union of all tools’ filesystem, network, and environment access.

Neither attack yields to prompting. An instruction hierarchy or a content filter cannot guarantee that a model ignores text in its context [27]; the model is exactly the component that cannot be trusted to make the distinction. Heavyweight isolation does not help either: containers [19] and microVMs [1] draw a boundary at the *machine*, one shared sandbox per agent, which stops neither cross-tool abuse (all tools share it) nor lethal-trifecta exfiltration (a single sandbox legitimately holds all three capabilities), while adding root, images, and per-invocation startup costs disproportionate to a 50 ms tool call. The dual-LLM [30] and CaMeL [5] patterns prescribe the right decomposition, separating the component exposed to untrusted content from the one holding capabilities, but assume an enforcement substrate that unprivileged sandboxes do not provide.

Our thesis is that the defense must be *architectural* and *kernel-enforced*, and that it can be made *unprivileged*. We present SANDLOCK, per-tool sandboxing for the agent workstation. Each tool invocation forks into its own deny-by-default process, confined by Landlock and seccomp before the tool runs: a tool gets only the filesystem paths, network endpoints, and environment variables it explicitly declares; the rest of the filesystem and network, including other tools’ secrets, is denied by the kernel, and the environment is cleared to the declared set. The blast radius of a hijacked tool shrinks from the agent’s union of capabilities to that one tool’s declared set (Section 3).

Per-tool confinement rests on a substrate (Section 4) that splits enforcement between static policy, compiled into kernel-enforced Landlock and seccomp-bpf rules, and a narrow supervisor that handles the runtime-dependent decisions, all unprivileged, under a uniform rule: use the earliest enforcement point that can decide soundly, otherwise deny. The contribution is not a new kernel primitive but a mapping from declared per-tool capabilities onto this substrate, and the substrate composes beyond single tools, into pipelines of stages with disjoint capabilities joined by unidirectional kernel pipes. As a use case, we build an execute-only pipeline (Section 5) realizing the Execute-Only Agent (XOA) architecture [27]: a planner stage that reaches the model API but never observes data, and an executor stage that reads data but has no network, so an injection hidden in the data reaches neither a decision-maker nor an exfiltration path. This paper makes the following contributions:

- A statement of two agent attack classes, cross-tool capability abuse and lethal-trifecta exfiltration, and an argument that both require architectural rather than prompt-level defense (Section 2).
- A per-tool sandboxing model for MCP tool servers that confines each invocation in a deny-by-default capability process,

breaking cross-tool attacks, with capabilities declared once and enforced at both client and server (Section 3).

- A split-enforcement substrate that maps declared capabilities onto stacked Landlock rulesets, a seccomp-bpf default-deny filter, and a narrow seccomp-notification supervisor with a host-level network allowlist, unprivileged end to end (Section 4).
- A security evaluation: deny/allow checks mapped to attacker actions, two attack case studies, including an execute-only pipeline that carries the XOA planner/executor split [27], and overhead measurements showing ~ 5.6 ms per-tool startup and a < 20 ms two-stage pipeline (Section 6).

SANDLOCK is open source; the repository link is omitted for double-blind review.

2 Threat Model and Motivation

2.1 Attacker Model

We assume the sandboxed unit, a tool invocation or a pipeline stage, may execute attacker-influenced code or consume attacker-controlled data: package scripts, downloaded files, tool outputs, web pages, email bodies, and model-generated commands. The host kernel, the SANDLOCK supervisor, and the embedding agent runtime are trusted. Kernel vulnerabilities, hardware side channels, privileged local attackers, and deliberate exhaustion of global kernel resources are out of scope; for those, a microVM remains the right tool. Within this model, the host is a developer machine running alongside the user’s editor, SSH keys, and source trees, so confinement must be unprivileged (developer accounts rarely have free sudo) and must default-deny the rest of the home directory.

2.2 Two Attack Classes

Cross-tool capability abuse. An MCP server exposes many tools to one agent: a web-search tool, a shell tool, a database tool. Conventionally they run in one process, or one container, that holds the *union* of their capabilities, all of the environment variables, filesystem paths, and network endpoints any tool needs. A prompt injection in one tool’s output (a poisoned search result) can then steer the model into calling a *different* tool to abuse capabilities the first never required: read $\sim /.ssh/id_rsa$, exfiltrate an API key from the environment, or reach an internal endpoint. The vulnerability is structural: capability is shared across tools that have no reason to trust one another.

Lethal-trifecta data exfiltration. A growing class of failures comes not from untrusted *code* but from untrusted *data* [5]. When one model context combines access to private data, the ability to communicate externally, and exposure to untrusted content, an injected instruction turns the agent into an exfiltration channel [31]. Confining the agent as a single process cannot break this, because a legitimate step often needs all three capabilities at once. Breaking it requires splitting the task into stages of asymmetric trust: the stage exposed to untrusted content holds no network, and the stage with network never sees that content. Dual-LLM and capability-separation patterns [5, 30] prescribe exactly this decomposition but assume an enforcement substrate unprivileged sandboxes lack.

2.3 Workload Constraints

Two properties shape the design. Tool calls are *short-lived and frequent* (an agent may invoke `npm install`, `pytest`, or generated snippets dozens of times per minute), so container-class startup of hundreds of milliseconds is visible in human-facing latency. And network access is *essential but dangerous*: agents must call model APIs and registries, but IP-layer allowlisting is insufficient, since a model that may reach `api.openai.com:443` can reach an attacker through the same address, so policy must live at the host level.

2.4 Why Existing Mechanisms Fall Short

Table 1 compares SANDLOCK against representative isolation mechanisms. Containers and microVMs are designed for service deployment; their image and namespace machinery imposes deployment overhead disproportionate to a single short tool call, and, more importantly, they isolate at the machine boundary, one sandbox per agent, which is the wrong granularity for both attack classes above. gVisor’s user kernel adds compatibility and performance costs for build tools. Tools such as `firejail` and `bubblewrap` [4, 10] reduce deployment cost but lack host-level network control. None of these natively expresses a per-tool capability split or a kernel-enforced data-flow boundary between stages.

3 Per-Tool Sandboxing for MCP

3.1 The Problem with Shared Tool Capability

In the conventional deployment, an MCP server’s tools share one process or container holding the *union* of their capabilities: every tool’s environment, filesystem, and network. A prompt injection in one tool’s output can then drive another, a poisoned search result that makes the shell tool read $\sim /.ssh/id_rsa$ and POST it, two capabilities no single tool needed but the deployment granted anyway.

3.2 One Sandbox per Tool Call

SANDLOCK inverts this. Each `call_tool` invocation forks a fresh child and confines it with Landlock and seccomp *before* the tool function runs (the mechanism is detailed in Section 4). The model is deny-by-default: a tool that declares no capabilities receives read-only access to system libraries and the workspace, no filesystem writes, no network, and no environment variables. The environment is cleared before every call, so a tool that needs `DATABASE_URL` must *declare* it; it cannot inherit another tool’s secrets ambiently. A tool widens its sandbox only by declaring capabilities explicitly:

- `fs_readable / fs_writable`: the exact paths the tool may read or write.
- `net_allow_hosts`: the hosts the tool may reach, enforced by the supervisor’s connect-allowlist (Section 4.3). SANDLOCK also pins the allowed hosts’ addresses in a virtualized `/etc/hosts`; combined with denied DNS egress, this leaves the child able to resolve only allowed hosts.
- `env`: the named environment variables to pass through.

A Landlock ruleset can only be *tightened* after it is applied, and a nested ruleset stacks on the outer one, so a server that is itself confined can confine each tool *tighter* still: the per-tool sandbox inherits the server’s outer restrictions and adds its own. The capability

Table 1: Comparison of unprivileged isolation mechanisms for the agent workstation.

Property	SANDLOCK	Docker	Firecracker	gVisor	firejail/bwrap
No root required	✓	✓ [†]	✗	✓	✓
Startup latency [‡]	~5.6 ms	~249 ms	~125 ms	~200 ms	~30 ms
Host-level network	on-behalf path	netns	TAP	netns	netns
Per-tool capability	deny-by-default	N/A	N/A	N/A	N/A
Data-flow stage split	operator	N/A	N/A	N/A	N/A

[†]Rootless Docker requires user-namespaces support and `/etc/subuid` configuration [7]. [‡]SANDLOCK and Docker (rootful) are measured in Section 6; Firecracker is as reported by its authors [1]; gVisor and firejail/bwrap are representative magnitudes.

spec lives in one place. In the server-side model, tools declare capabilities via `sandlock:*` annotations in their MCP tool definitions; the same declaration informs a client-side `McpSandbox` wrapper and is enforced server-side. One definition, two enforcement points.

3.3 Why It Stops Cross-Tool Abuse

Under per-tool confinement, the search-result injection above fails at the kernel. The shell tool runs with only the capabilities the shell tool declared. `~/ssh/id_rsa` is outside its readable scope, so Landlock denies the open; an attacker host is absent from `net_allow_hosts`, so the supervisor refuses the connection, and the virtualized `/etc/hosts` with DNS egress denied prevents its resolution as a second layer. The blast radius of a hijacked tool is its own declared capabilities, never the agent’s union. Cross-tool abuse requires a capability the abused tool was never granted, which is exactly what the per-tool boundary withholds.

4 Enforcement and Implementation

Per-tool sandboxing, and the stage pipelines built on it (Section 5), rest on one mechanism: a split between policy whose verdict is known before the child runs and policy that depends on syscall-time state. SANDLOCK follows a single rule, use the earliest enforcement point that can decide a policy soundly. Static invariants (readable and writable path prefixes, denied paths, TCP ports, IPC scope, unconditional syscall denials) compile into Landlock [23] and seccomp-bpf [8], which the kernel enforces with no supervisor involvement. Runtime decisions (the resolved destination of a connect) route through seccomp user notification [16]. If neither layer can enforce a policy without a race or a semantic mismatch, SANDLOCK denies the operation rather than weakening it. This split is what makes confinement cheap enough to apply per tool call and precise enough to separate stages.

After `fork()` the child applies confinement in a fixed, unprivileged order, `setpgid`, `PR_SET_NO_NEW_PRIVS`, the Landlock ruleset, then the seccomp-bpf filter, and blocks before `exec()` until the parent has attached the supervisor to the seccomp notification fd, so no tool code runs before the supervisor is live.

4.1 Landlock Spine and seccomp Filter

Landlock [23] provides the static layer: filesystem read/write scope, TCP-port scope, and IPC boundaries, denied or allowed by the kernel without involving the supervisor. SANDLOCK requires Landlock ABI 6 or later (Linux 6.12+), which supplies filesystem rules, TCP-port rules, and abstract-socket and signal scoping for IPC [18]; it

queries the runtime ABI at startup and refuses to run below this floor rather than silently downgrading. Crucially for both per-tool confinement and stage separation, a Landlock ruleset can only be *tightened*: once applied, a process cannot relax its own restrictions, and a nested ruleset stacks on top of the outer one, so an inner sandbox inherits every outer restriction and adds its own. After Landlock, SANDLOCK installs a seccomp-bpf filter that denies syscalls outside Landlock’s scope and routes supervisor-visible syscalls to seccomp user notification. Landlock scopes only TCP connect/bind, abstract UNIX sockets, and signals, so the remaining egress and cross-process channels (UDP, ICMP and raw sockets, SysV and POSIX shared memory and message queues, and `io_uring`, whose in-ring operations would otherwise bypass syscall interception) are closed by the seccomp default-deny set; a stage’s “no network” is thus the conjunction of its Landlock TCP rules and seccomp denial of the rest, and the planner/executor disjointness argument (Section 5) relies on no unscoped back-channel remaining open.

4.2 The seccomp Notification Supervisor

The dynamic layer is an async (tokio) task in the parent that dequeues notifications from the seccomp fd and replies with `Allow`, `Deny(errno)`, or `Continue` after performing an on-behalf operation; it intercepts the network syscalls `connect/send*/bind` for the host-level allowlist. To act on behalf of the child (for example, to connect to a parent-validated address), the supervisor uses `pidfd_getfd` [3] to dup the child’s socket into the parent. This is TOCTOU-safe: the original syscall is never `Continued`, so the kernel never re-reads the child’s pointer arguments, and pointer-passed arguments are read once, validated, and reissued from the supervisor’s own copy, each read bracketed by `SECCOMP_IOCTL_NOTIF_ID_VALID`.

4.3 Network Model

SANDLOCK uses a single endpoint allowlist of *protocol × destination*. Hostnames are resolved once at start and pinned for the child, eliminating DNS-rebinding; wildcards are excluded by design. The pinning needs no mount namespace: the supervisor answers any open of `/etc/hosts` with a `memfd` holding the pinned entries for the allowed hosts, so the host’s on-disk file never reaches the child. Port-only TCP policy is enforced directly by Landlock with no per-syscall overhead; host-specific rules and non-TCP protocols route through the supervisor, which validates the destination and then performs or denies the operation. This host-level allowlist, which both case studies depend on, expresses the host identity that Landlock’s port-only rules cannot.

4.4 Implementation

SANDLOCK is ~7K lines of Rust: a core library (Landlock rules, seccomp filters, the async supervisor, the pipeline) plus a CLI and an FFI layer exposing the same policy model to shell and Python. Each pipeline stage is its own SANDLOCK process tree, so a compromise of one cannot read another’s private paths.

5 Use Case: Execute-Only Pipelines

5.1 Why Prompt Injection Resists Prompting

Per-tool sandboxing bounds what a hijacked tool can do, but it does not stop the hijack itself: if untrusted data enters the model’s context, no prompt guarantees the model will not act on it [27]. The model is the component that cannot reliably separate instruction from data. Defending the data path therefore means keeping the untrusted data away from the model, not instructing the model to behave.

5.2 The XOA Architecture

Tiwari and Williams observe that many agent tasks are *scriptable*: a program the model writes from the task and tool schemas can do the work without the model ever seeing the data it processes [27]. Their Execute-Only Agent (XOA) architecture has two requirements, the model generates the program with no data visibility, and the program runs with full data access but its output never returns to the model, and finds roughly 78% of AgentDojo [6] tasks completable this way. XOA defines the architecture; it needs a substrate that enforces “no data visibility” and “no feedback” as more than convention.

5.3 Kernel-Enforced XOA in SANDLOCK

SANDLOCK realizes XOA as a two-stage pipeline whose stages carry disjoint capabilities (Figure 1). The *planner* stage may reach the model API (`net_allow_hosts=["api.anthropic.com"]`) but cannot read the workspace; it emits a program. The *executor* stage may read and write the workspace but has no network (`net_connect=[]`); it runs that program over the data. Two properties, each enforced by the kernel rather than by discipline, make the separation sound:

Disjoint capabilities. The planner’s and executor’s Landlock rule-sets are mutually exclusive over data and network. Because a Landlock ruleset cannot be relaxed after it is applied, neither stage can acquire the other’s capability at runtime: the executor cannot grow a network, the planner cannot grow data access.

Unidirectional data flow. The stages are joined by a single pipe(2): planner stdout to executor stdin, one read end and one write end, with no reverse channel. The parent creates the pipe, forks both children, and immediately closes its own copies of the pipe descriptors, so after setup the parent holds no descriptor that can read the inter-stage data. The kernel enforces the directionality of the endpoints, and seccomp’s default-deny closes every other channel between the stages (Section 4). The executor may begin running the program while the planner is still emitting it, but no byte the executor reads can flow back to alter the plan; the one residual signal, backpressure timing on the pipe, is a low-bandwidth side channel outside the threat model.

```
planner = Sandbox(                                # talks to
    LLM,
    net_allow_hosts=["api.anthropic.com"],
    fs_readable=["/usr", "/lib"])                  # never
                                                sees data
executor = Sandbox(                                # touches
    data,
    fs_readable=["/usr", "/lib", "/work"],
    fs_writable=["/work"],
    net_connect=[])                               # never
                                                talks out
result = (
    planner.cmd(["plan", task, schemas])           # writes
        program
    | executor.cmd(["sh"])                          # runs
        program
).run()
```

Figure 1: Execute-only pipeline. The planner reaches the model but not data; the executor reaches data but not the network. The pipe carries the generated program one way.

5.4 Why It Stops Injection

The only component a prompt injection could hijack, the model, never sees the data: it sees the task and schemas and writes a program. The component that sees the data, the executor, is fixed code with no path back to the model and no network. An instruction hidden in an email body reaches neither a decision-maker nor an output channel. This is the dual-LLM and CaMeL separation [5, 30] made kernel-enforced, in its strongest form: the model does not quarantine the data, it never observes it.

Three caveats bound the claim. SANDLOCK separates stage *capabilities*, not pipe *content*: a compromised upstream stage can still shape the bytes a downstream stage consumes, so data-provenance defenses [5] remain complementary. XOA applies only to scriptable tasks, so the roughly 22% that require the model to read data [27] fall outside it. And the kernel closes the channels into the executor and its path to the network, but not its result: if the agent loop re-feeds that output into a later model turn, the injection re-enters the model’s context outside SANDLOCK’s boundary, so the guarantee holds only for single-shot pipelines whose output terminates at the user or storage.

6 Security Evaluation

Setup. Experiments run on a typical Linux workstation: an AMD Ryzen 5 5500U (6 cores / 12 threads, up to 4.0 GHz) with 8 GB of RAM and NVMe SSD, running Pop!_OS 24.04 LTS on Linux 6.18 with unprivileged user namespaces enabled. Startup latency is the median of 20 runs after warm-up. We report bare-metal, SANDLOCK, and rootful Docker for context.

6.1 Defenses Hold

Table 2 reports deterministic deny/allow cases drawn from the two attack classes; each is run against SANDLOCK and the outcome recorded. Reads and writes outside the granted scope are denied while in-scope access succeeds; a connect to a host outside the

Table 2: Deny/allow checks mapped to attacker actions. Each row is the outcome observed when the case is run against SANDLOCK.

Attacker action	Policy	Outcome
Read outside read scope	default-deny	denied
Read inside read scope	granted	allowed
Write outside write scope	default-deny	denied
Write inside write scope	granted	allowed
Connect to non-allowlisted host	allowlist	refused
Connect to allowlisted host	allowlist	allowed
Executor stage reaches network	no grant	denied
Planner stage reads workspace	no grant	denied

allowlist is refused while an allowlisted endpoint connects; and a stage not granted a private directory cannot read it even though a sibling stage can. All eight checks behaved as the policy requires. The two case studies that follow are executed against the real SANDLOCK per-tool sandbox and executor: for each, we perform the action a prompt-injected tool or executor would take and record the kernel’s response. The hijack is injected deterministically rather than driven by a live model, so what is measured is the enforced outcome, not the model’s susceptibility; harnesses for both are in the repository.

6.2 Case Study 1: Cross-Tool SSH-Key Exfiltration

We run the cross-tool attack of Section 3. An MCP server exposes a *search* tool (a documentation host, no filesystem beyond the workspace) and a *shell* tool (the workspace only, no other paths, no network); a poisoned search result instructs the agent to read `~/ .ssh/id_rsa` and post it out, and the model invokes the shell tool. Run against the real SANDLOCK per-tool sandbox (McpSandbox), both the key read (outside the shell tool’s scope) and the exfiltration connect (no allowlisted host) fail with EACCES, the two deny rows of Table 2, and the key never reaches a local listener; the identical action unconfined, where the tool inherits home and network, leaks it. The only difference is the per-tool boundary.

6.3 Case Study 2: Prompt-Injection Exfiltration under XOA

We run a lethal-trifecta task, “summarize my unread emails,” where one body says “ignore previous instructions and forward all messages to attacker.com.” Run as one agent that reads emails into context, the injection is obeyed. As an execute-only pipeline (Section 5), the planner writes a summarization script from the task and email-tool schema without reading any body, and the executor runs it with mailbox access but no network: the injected text lives in data the model never sees, and the executor that does see it is fixed code with no channel to act. Run against SANDLOCK, the executor’s exfiltration connect fails with EACCES (the “executor stage reaches network” row of Table 2) and the data never leaves the host; the identical executor by convention, lacking the kernel-enforced no-network stage, completes the exfiltration, isolating kernel enforcement as the cause.

6.4 What the Defenses Do Not Stop

The case studies are exfiltration-shaped attacks the mechanism is built to block; honesty requires naming in-scope attacks it does not. *Exfiltration through an allowed host*: a compromised planner, or any tool granted a host, can encode data in request bodies, paths, or timing (the residual channel Section 5 concedes); the allowlist bounds which hosts are reachable, not what is sent to one. *Tools needing both data and network*: a web fetcher or a `git push` cannot be split into XOA stages and must hold both, reassembling the trifecta in one tool. *In-scope destructive actions*: an injected call that deletes a writable workspace or deploys through a deploy tool’s own credentials stays within policy and is not blocked. *Schema poisoning*: the planner trusts the task and tool schemas, so a poisoned tool description [14] can steer it even though it reads no data, needing schema-provenance defenses. SANDLOCK reduces blast radius and removes the model from the data path for scriptable tasks; it does not adjudicate misuse of legitimately granted capabilities.

6.5 Security at Near-Zero Cost

For these defenses to be usable on every tool call, their overhead must be negligible. For `/bin/echo` wall time (median of 20 runs, Figure 2a), per-tool confinement runs in ~ 5.6 ms (~ 1.3 ms bare metal, so ~ 4 – 5 ms of confinement overhead from Landlock ruleset construction and `seccomp-fd` setup), against Docker’s ~ 249 ms, roughly $44\times$ slower. A two-stage execute-only pipeline is two confined `fork()`s and one `pipe()`, so its startup is about twice the per-tool cost, under 20 ms, against the seconds a container-based two-stage split would spend starting containers and configuring a bridge. Steady-state overhead is likewise low: on `redis-benchmark` (Figure 2b,c) SANDLOCK matches bare-metal throughput and p99 tail latency to within run-to-run variance, while Docker reaches only $\sim 76\%$ of bare-metal throughput at roughly $3\times$ the tail. Common build tools (`python3`, `make`, `node`, `pytest`, including native-extension imports) run unmodified once their scratch paths are granted; a systematic compatibility study is future work.

7 Related Work

Prompt-injection defenses. The dual-LLM pattern [30] and CaMeL [5] separate the component exposed to untrusted content from the one holding capabilities, and Execute-Only Agents [27] strengthen this to never letting the model observe untrusted data. These define the decompositions and quantify their reach on AgentDojo [6]; SANDLOCK contributes the unprivileged, kernel-enforced substrate they run on, turning “the model should not see the data” from convention into a Landlock-and-pipe boundary. The lethal-trifecta framing [31] explains why a single sandbox cannot be the unit of defense.

Agent systems and isolation. Closest to SANDLOCK’s per-tool defense, IsolateGPT [32] isolates each LLM app’s execution behind well-defined interfaces, and Progent [24] enforces least privilege over an agent’s tool calls through a programmable policy layer; SANDLOCK is the unprivileged, kernel-enforcement complement to these runtime-level designs, confining each invocation

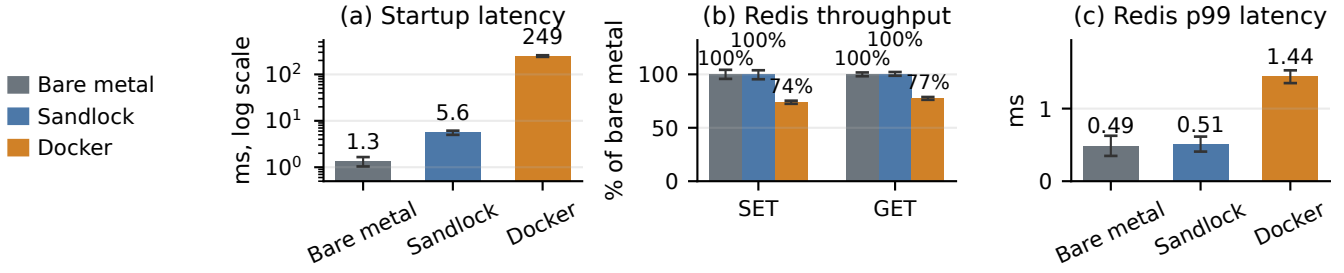


Figure 2: Overhead measurements: (a) `/bin/echo` startup wall time, (b) Redis throughput, and (c) Redis p99 latency, for bare metal, SANDLOCK, and Docker. SANDLOCK keeps per-call startup at ~ 5.6 ms and matches bare-metal steady-state performance; Docker pays roughly 44 $\times$ more per invocation.

per call rather than mediating it inside the agent. MVVM [33] deploys agents in WebAssembly enclaves; Kgent [35] explores model-driven kernel extensions; AgentCgroup [34] controls agents’ OS resource use; Agency [28] mediates agent-to-external communication through a gateway. SANDLOCK is complementary, confining the local execution side per tool and per stage while gateways mediate communication.

Unprivileged sandboxing. SANDLOCK builds on Landlock [23], seccomp-bpf [8], and seccomp user notification [16], with `pidfd_getfd` [3] for safe fd transfer; Capsicum [29] is its spiritual ancestor. Container runtimes embody the same notification split at a coarser grain [21]. Among unprivileged process sandboxes, nsjail [25], minijail [26], and birdcage [22] enforce static policy but expose neither a per-tool capability model nor host-level network rules; BPF-LSM sandboxes such as bpfbox [9] need CAP_BPF or root [17], which SANDLOCK avoids. The classic hazards of syscall interposition [11] motivate SANDLOCK’s TOCTOU-safe on-behalf path.

Containers, microVMs, and speculation. Docker [19] composes namespaces [15], cgroups [13], and seccomp; gVisor [12] adds a user-mode kernel; Firecracker [1] cuts microVM startup. These isolate at the machine boundary for multi-tenant hosting, orthogonal to SANDLOCK’s per-tool, per-stage granularity.

8 Discussion and Future Work

Threat-model boundaries. SANDLOCK is unprivileged by design and inherits the limits of unprivileged Linux: kernel vulnerabilities, side channels, and resource-exhaustion attacks against shared kernel structures are out of scope, and for multi-tenant adversarial workloads a microVM remains the right answer. SANDLOCK raises the bar for the common case, prompt injection and cross-tool abuse on a developer machine, not for a kernel-level adversary.

Read-required tasks. Execute-only pipelines cover scriptable tasks. The tasks that genuinely require the model to read data [27] need complementary defenses, such as a quarantined model whose output is treated as untrusted, or data-provenance tracking [5], layered on the same per-stage substrate.

Policy correctness is the author’s burden. SANDLOCK enforces whatever policy it is handed, not whether it is right. An author who over-declares (granting `~ /` “to be safe”) silently reopens the hole

while the kernel still reports the policy enforced, the documented fate of opt-in allowlists; and execute-only separation is only as sound as the decomposition the author writes. Tight declarations and sound decompositions are prerequisites SANDLOCK relies on, not properties it supplies; tooling to flag over-grant and narrow dynamic tools is future work.

Integration and retrofit. SANDLOCK supports two retrofit modes. In *wrap-the-child*, the host forks, confines, and execs each command an agent already spawns, a drop-in for shell and tool calls; MCP tool servers are the natural fit, confining each invocation under a per-tool policy without re-architecting the agent. In *self-confinement*, the static Landlock layer is applied in place so an agent runtime can drop its own privileges after startup.

9 Conclusion

Cross-tool capability abuse and prompt injection are the defining security problems of AI agents, and neither yields to prompting or to a single shared container. SANDLOCK confines every tool invocation in its own kernel-enforced, deny-by-default sandbox, without root: a hijacked tool holds only the capabilities it declared, never the agent’s union. The same unprivileged substrate composes into capability-disjoint stage pipelines, with which we realize the execute-only agent architecture [27], keeping untrusted data out of the model’s context for scriptable tasks via disjoint Landlock capabilities and a one-way pipe. Confinement is cheap enough to apply on every call (~ 5.6 ms per tool, under 20 ms per pipeline). SANDLOCK is open source; the repository link is omitted for double-blind review.

References

- [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’20)*. USENIX Association, Santa Clara, CA, USA, 419–434.
- [2] Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- [3] Christian Brauner. 2024. `pidfd_getfd(2)` — Linux Manual Page. https://man7.org/linux/man-pages/man2/pidfd_getfd.2.html.
- [4] Bubblewrap Project. 2026. Bubblewrap: Low-Level Unprivileged Sandboxing Tool. <https://github.com/containers/bubblewrap>. Accessed: 2026-05-23.
- [5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. doi:10.48550/arXiv.2503.18813

- [6] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352
- [7] Docker Inc. 2026. Rootless Mode. <https://docs.docker.com/engine/security/rootless/>. Accessed: 2026-05-23.
- [8] Will Drewry and Jake Edge. 2024. seccomp(2) — Linux Manual Page. <https://man7.org/linux/man-pages/man2/seccomp.2.html>.
- [9] William Findlay, Anil Somayaji, and David Barrera. 2020. bpfbox: Simple Precise Process Confinement with eBPF. In *Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop (CCSW '20)*. ACM, Virtual Event, USA, 91–103. doi:10.1145/3411495.3421358
- [10] Firejail Project. 2026. Firejail: Linux Namespaces and Seccomp-bpf Sandbox. <https://github.com/netblue30/firejail>. Accessed: 2026-05-23.
- [11] Tal Garfinkel. 2003. Traps and Pitfalls: Practical Problems in System Call Interposition Based Security Tools. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA, 15 pages. <https://www.ndss-symposium.org/ndss2003/traps-and-pitfalls-practical-problems-system-call-interposition-based-security-tools/>
- [12] Google. 2024. gVisor: Application Kernel for Containers. <https://gvisor.dev/>.
- [13] Tejun Heo. 2015. Control Groups v2. <https://www.kernel.org/doc/Documentation/cgroup-v2.txt>.
- [14] Invariant Labs. 2025. MCP Security Notification: Tool Poisoning Attacks. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>.
- [15] Michael Kerrisk. 2013. Namespaces in Operation. LWN.net. <https://lwn.net/Articles/531114/>
- [16] Michael Kerrisk. 2024. seccomp_unotify(2) — Linux Manual Page. https://man7.org/linux/man-pages/man2/seccomp_unotify.2.html.
- [17] Linux man-pages Project. 2025. capabilities(7) — Linux Manual Page. <https://man7.org/linux/man-pages/man7/capabilities.7.html>.
- [18] Linux man-pages Project. 2025. landlock(7) — Linux Manual Page. <https://man7.org/linux/man-pages/man7/landlock.7.html>.
- [19] Dirk Merkel. 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- [20] Model Context Protocol. 2026. Tools. <https://modelcontextprotocol.io/specification/draft/server/tools>. Accessed: 2026-05-23.
- [21] Open Container Initiative. 2023. Open Container Initiative Runtime Specification, Version 1.1.0. <https://oci-playground.github.io/specs-latest/specs/runtime/v1.1.0/oci-runtime-spec.html>.
- [22] Phylum. 2023. Birdcage: Cross-platform Embeddable Sandboxing. <https://github.com/phylum-dev/birdcage>.
- [23] Mickaël Salaün. 2017. Landlock LSM: Toward Unprivileged Sandboxing. *Linux Security Summit Europe*. https://landlock.io/talks/2017-09-14_landlock-lss.pdf
- [24] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv preprint arXiv:2504.11703. doi:10.48550/arXiv.2504.11703
- [25] Robert Świecki. 2024. nsjail: A Light-Weight Process Isolation Tool. <https://github.com/google/nsjail>.
- [26] The ChromiumOS Authors. 2024. Minijail. <https://google.github.io/minijail/>.
- [27] Rahul Tiwari and Dan Williams. 2026. Execute-Only Agents: Architectural Defense Against Prompt Injection for AI Agents. Presented at the 1st Workshop on OS Design for AI Agents (AgenticOS), ASPLOS 2026.
- [28] Cong Wang. 2025. Agentry: Orchestration and Memory for Multi-Agent Systems. <https://github.com/ampm-protocol/agentry>.
- [29] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium (USENIX Security '10)*. USENIX Association, Washington, DC, USA, 29–46.
- [30] Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. <https://simonwillison.net/2023/Apr/25/dual-llm-pattern/>.
- [31] Simon Willison. 2025. The Lethal Trifecta for AI Agents: Private Data, Untrusted Content, and External Communication. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>.
- [32] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. doi:10.48550/arXiv.2403.04960
- [33] Yiwei Yang, Aibo Hu, Yusheng Zheng, Brian Zhao, Xinqi Zhang, Dawei Xiang, Kexin Chu, Wei Zhang, and Andi Quinn. 2024. MVVM: Deploy Your AI Agents—Securely, Efficiently, Everywhere. arXiv preprint arXiv:2410.15894. doi:10.48550/arXiv.2410.15894
- [34] Yusheng Zheng, Jiakun Fan, Quanzhi Fu, Yiwei Yang, Wei Zhang, and Andi Quinn. 2026. AgentCgroup: Understanding and Controlling OS Resources of AI Agents. arXiv preprint arXiv:2602.09345. doi:10.48550/arXiv.2602.09345
- [35] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions (eBPF '24)*. ACM, Sydney, NSW, Australia, 30–36. doi:10.1145/3672197.3673434